

Welcome Offer Access to everything. Now 20% off. [Upgrade now](#)

Secure Ops + App Sec Pipeline + Soar +

ShieldFlow: Building an Automated AppSec Pipeline & SOAR Platform with Open-Source Security Tools



Expadox Lab

Follow

5 min read · Jun 7, 2026



22



1



A project completed by Expadox Lab Cohort 2, 2026 — Team ThreatMatrix

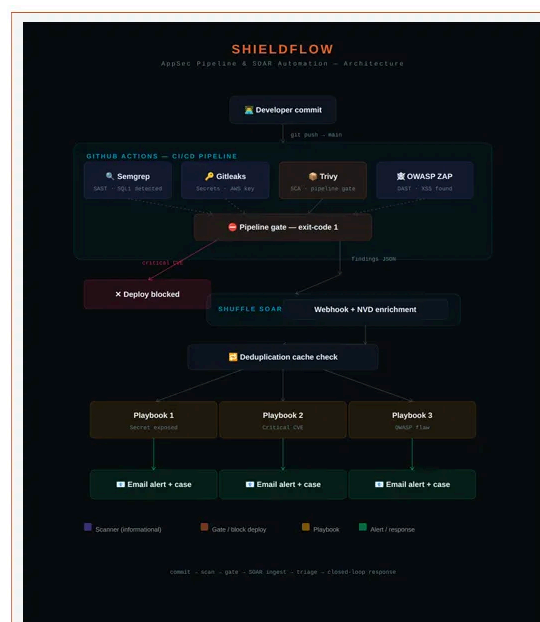


Figure 1: ShieldFlow Architecture — End-to-End AppSec Pipeline and SOAR Automation Workflow.

Team Members

Open in app ↗



Medium



Search



Write



- Joys K Matthew
- Umaru Abdulrahman
- Shadia Bello
- Esther Owah

Introduction

Modern software teams deploy code faster than ever, but security often struggles to keep pace. Vulnerabilities are frequently discovered after deployment, when remediation is costly and attackers may already have access.

To address this challenge, our team designed and implemented **ShieldFlow**, an end-to-end Application Security (AppSec) and Security Orchestration, Automation and Response (SOAR) platform that embeds security directly into the CI/CD pipeline.

ShieldFlow automatically scans code, dependencies, secrets, and running applications for vulnerabilities, then enriches findings and triggers

automated response workflows without requiring manual intervention.

The project was developed during Expadox Lab Sprint 2 and demonstrates how organizations can adopt DevSecOps practices using entirely open-source technologies.

Project Objective

The primary goal of ShieldFlow was to create a security-first software delivery pipeline capable of:

- Detecting vulnerabilities before deployment
- Preventing vulnerable applications from reaching production
- Automating security triage and alerting
- Reducing manual response time
- Demonstrating DevSecOps and SOAR principles in a practical environment

Unlike traditional security assessments that occur after deployment, ShieldFlow integrates security testing directly into the development lifecycle.

The Problem

Many organizations face several common security challenges:

- Hardcoded secrets accidentally committed to repositories
- Vulnerable third-party dependencies

- Injection vulnerabilities hidden within source code
- Lack of centralized security alert management
- Slow response to discovered vulnerabilities
- Alert fatigue caused by repeated notifications

Without automation, security teams often spend significant time manually reviewing findings, prioritizing risks, and notifying stakeholders.

ShieldFlow was designed to solve these problems through automation.

Architecture Overview

ShieldFlow combines four security scanners with a SOAR platform.

Application Security Layer

The AppSec pipeline consists of:

ToolPurposeSemgrepStatic Application Security Testing (SAST)GitleaksSecrets DetectionTrivySoftware Composition Analysis (SCA)OWASP ZAPDynamic Application Security Testing (DAST)

All scanners execute automatically through GitHub Actions whenever code is pushed to the repository.

SOAR Layer

Security findings are forwarded into Shuffle SOAR where they are:

- Ingested via webhooks

- Enriched with CVE information from the NVD API
- Routed to dedicated response playbooks
- Deduplicated to prevent alert fatigue
- Sent to stakeholders through automated notifications

This architecture enables security findings to move from detection to response in seconds rather than minutes.

Phase 1: Building the AppSec Pipeline

Target Application

We used DVWA (Damn Vulnerable Web Application) as the target application because it intentionally contains security flaws including:

- SQL Injection
- Cross-Site Scripting (XSS)
- Command Injection
- CSRF
- File Inclusion vulnerabilities

This allowed us to validate the effectiveness of multiple security testing approaches.

Figure 2: DVWA (Damn Vulnerable Web Application) used as the target application for security testing.

Static Application Security Testing (SAST)

Semgrep was integrated into GitHub Actions to analyze source code automatically.

To validate detection capabilities, a SQL Injection vulnerability was intentionally introduced into a Python application.

Semgrep successfully identified the vulnerable database query and generated SARIF-formatted findings for review.

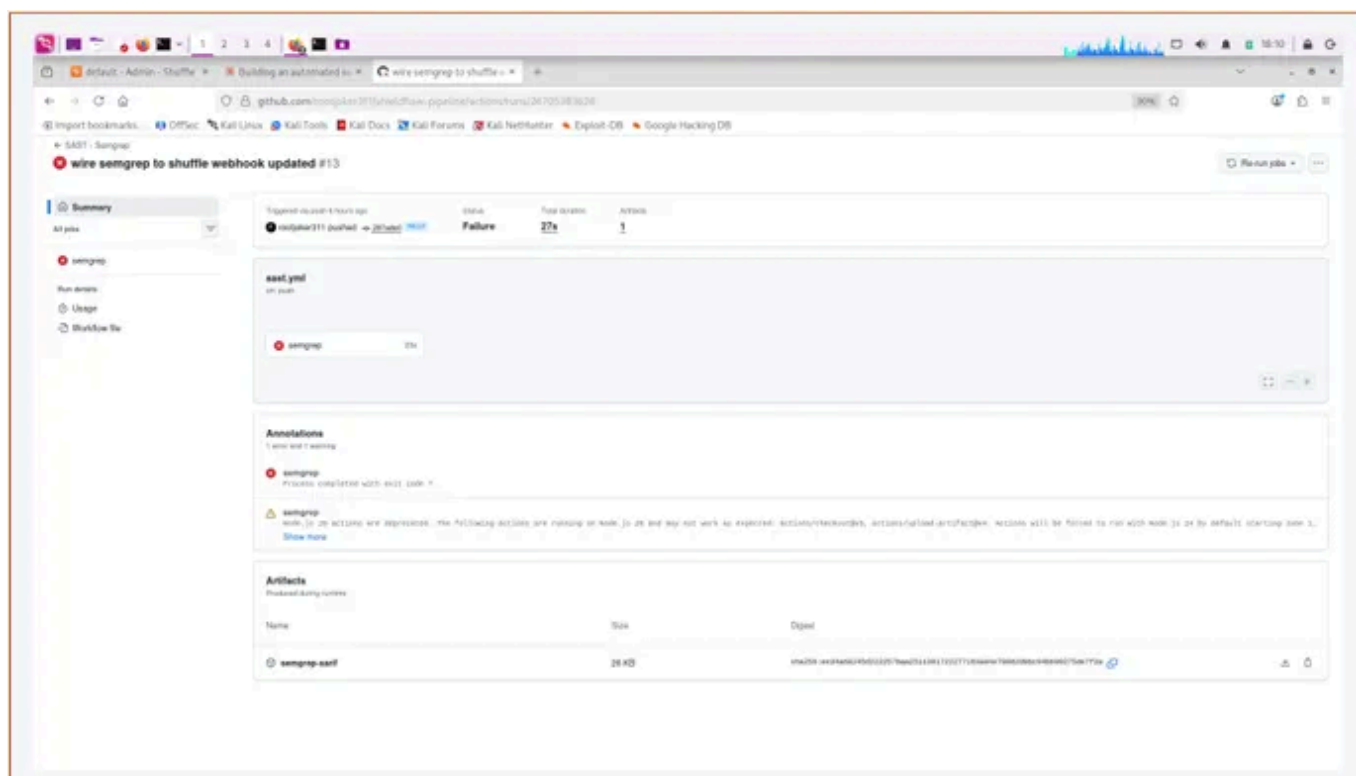
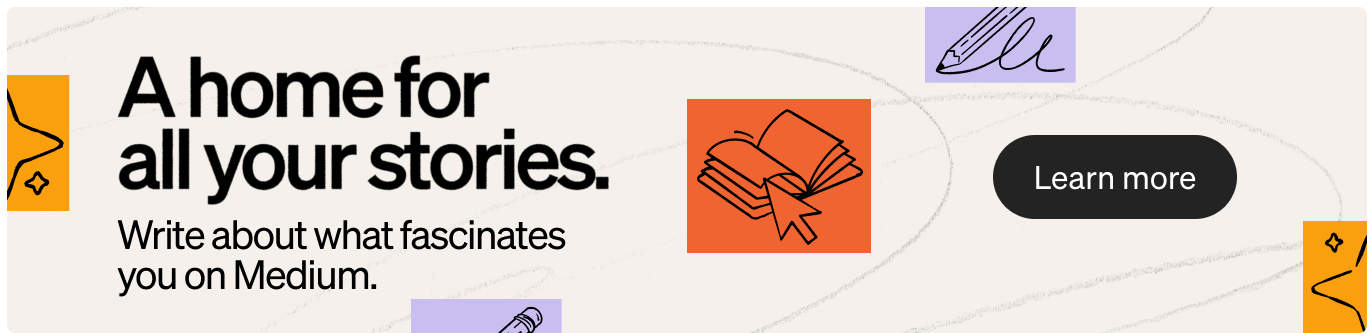


Figure 3: Semgrep detecting SQL Injection vulnerabilities during static code analysis.

Secrets Detection

Gitleaks was configured to scan the entire Git history for exposed credentials.



Test secrets included:

- AWS Access Keys
- AWS Secret Keys
- Database passwords

The scanner successfully detected exposed credentials and produced security findings for further processing.

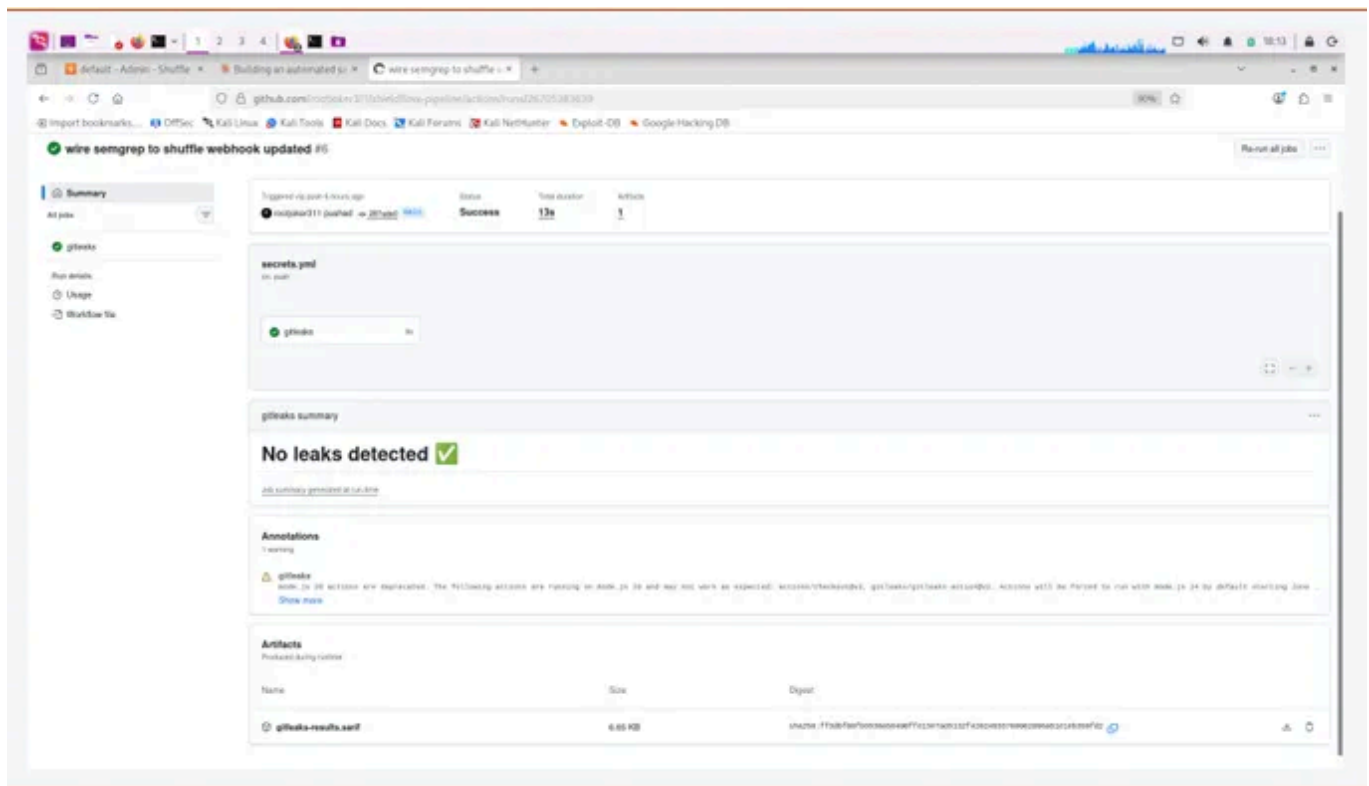


Figure 4: Gitleaks scanning repository history for exposed secrets and credentials.

Dependency Scanning

Trivy was deployed as the Software Composition Analysis (SCA) engine.

Outdated packages containing known CVEs were intentionally added to the environment.

Trivy identified multiple critical vulnerabilities and acted as the deployment gate by automatically failing the pipeline when critical risks were detected.

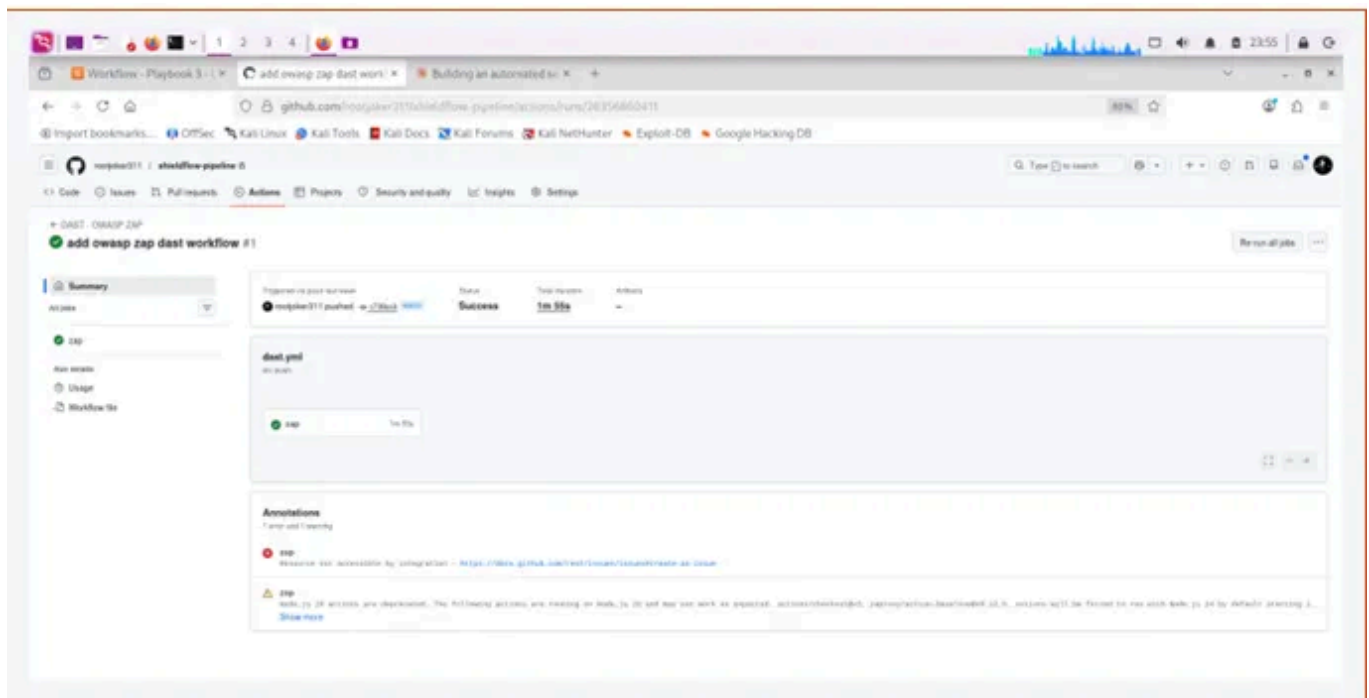


Figure 6: OWASP ZAP performing dynamic security testing against the running application.

Phase 2: Security Orchestration & Automated Response

After vulnerability detection, findings were automatically forwarded to Shuffle SOAR.

Automated Ingestion

A webhook-based ingestion workflow received findings from the security scanners and normalized the data for processing.

NVD API call configured in Http_1 node:

```
GET https://services.nvd.nist.gov/rest/json/cves/2.0?cveId=${Webhook_1.body.finding}}
```

The `${Webhook_1.body.finding}` variable dynamically inserts the CVE ID from the incoming payload.

Status note: The HTTP enrichment node is configured and wired into the workflow. During testing, live execution through Orborus was intermittent, so CVSS enrichment was also verified manually against the NVD API as a fallback:

```
curl -s "https://services.nvd.nist.gov/rest/json/cves/2.0?cveId=CVE-2019-14806" | python3 -m json.tool | grep -A5 cvssMetric
```

Figure 8: Automated vulnerability enrichment using the National Vulnerability Database (NVD).

Security Playbooks

Three dedicated response playbooks were developed:

Figure 9: Automated security playbooks developed in Shuffle SOAR for credential exposure, critical CVE detection, and OWASP vulnerability response.

Playbook 1 — Secret Exposure

- Detect exposed credentials
- Notify responsible stakeholders
- Create investigation cases

Playbook 2 — Critical CVE Detection

- Alert developers
- Communicate deployment-blocking vulnerabilities
- Provide remediation information

Playbook 3 — OWASP Findings

- Create security cases

- Notify AppSec personnel
- Attach supporting evidence

These playbooks reduced manual effort and accelerated incident handling.

Figure 10: Automated email notifications generated by ShieldFlow after security findings are detected and processed.

Key Challenges We Faced

Several technical challenges emerged during implementation:

GitHub Actions YAML Errors

Workflow files repeatedly failed due to formatting and indentation issues.

We solved this by generating workflow configurations programmatically using terminal commands instead of manual editing.

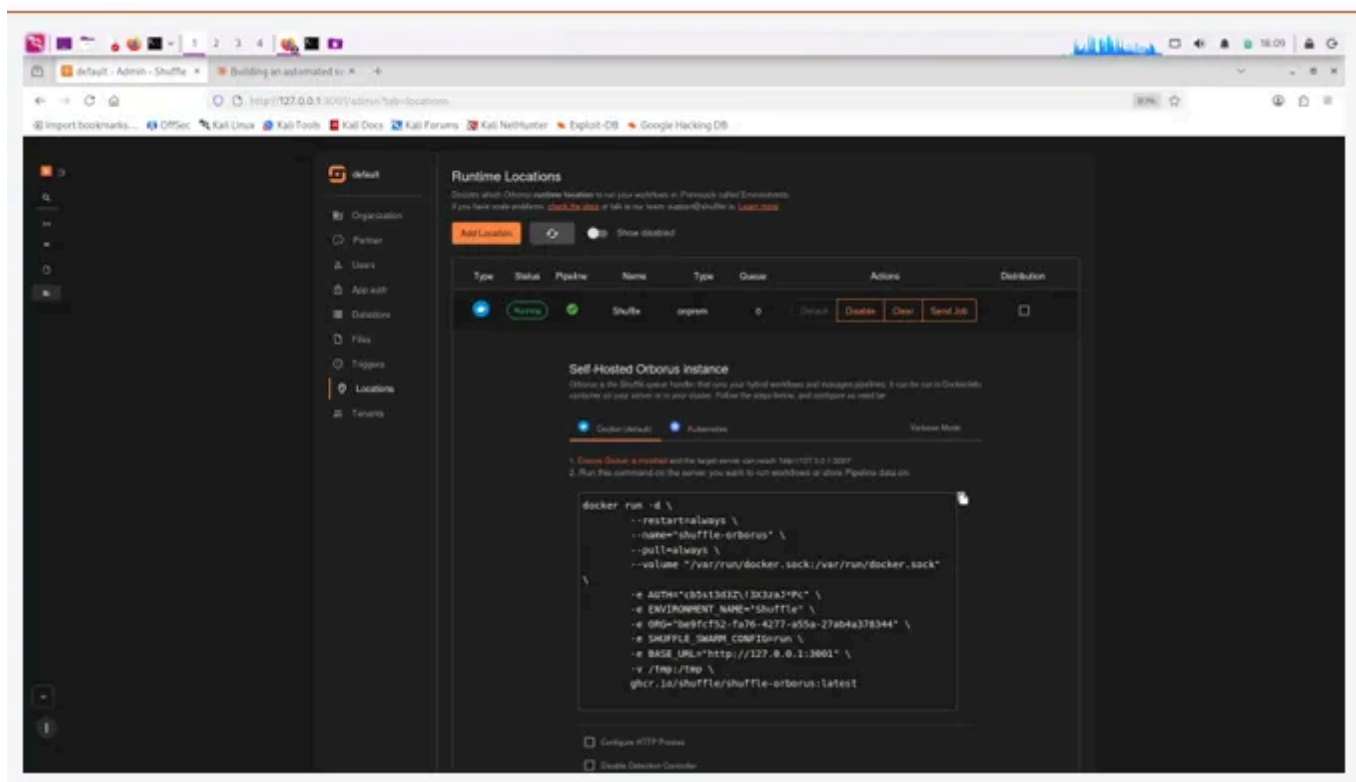


Figure 13: Successful Orborus deployment after resolving workflow execution issues.

Results

The project successfully identified **11 confirmed vulnerabilities**, including **4 critical findings**.

Figure 14: Eleven confirmed vulnerabilities identified during testing.

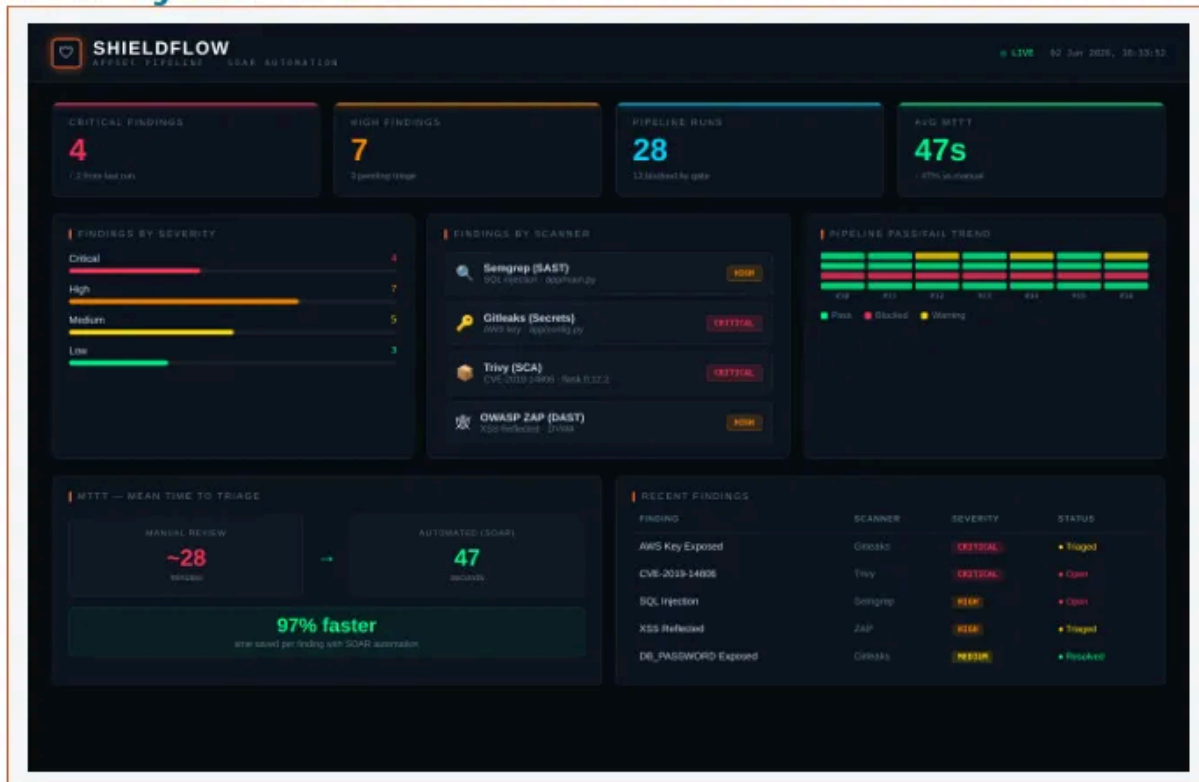
Examples included:

- SQL Injection
- Exposed AWS credentials
- Vulnerable dependencies
- Cross-Site Scripting

- Missing security headers

Most importantly, automated triage reduced Mean Time To Triage (MTTT) from approximately **28 minutes** to **47 seconds**, representing a **97% improvement**.

Live Security Dashboard



Evidence: ShieldFlow dashboard — KPIs (critical/high findings, pipeline runs, MTTT), findings by severity and scanner, pipeline pass/fail trend, MTTT comparison, and recent findings table.

Figure 15: ShieldFlow dashboard displaying security findings, pipeline metrics, and operational security insights

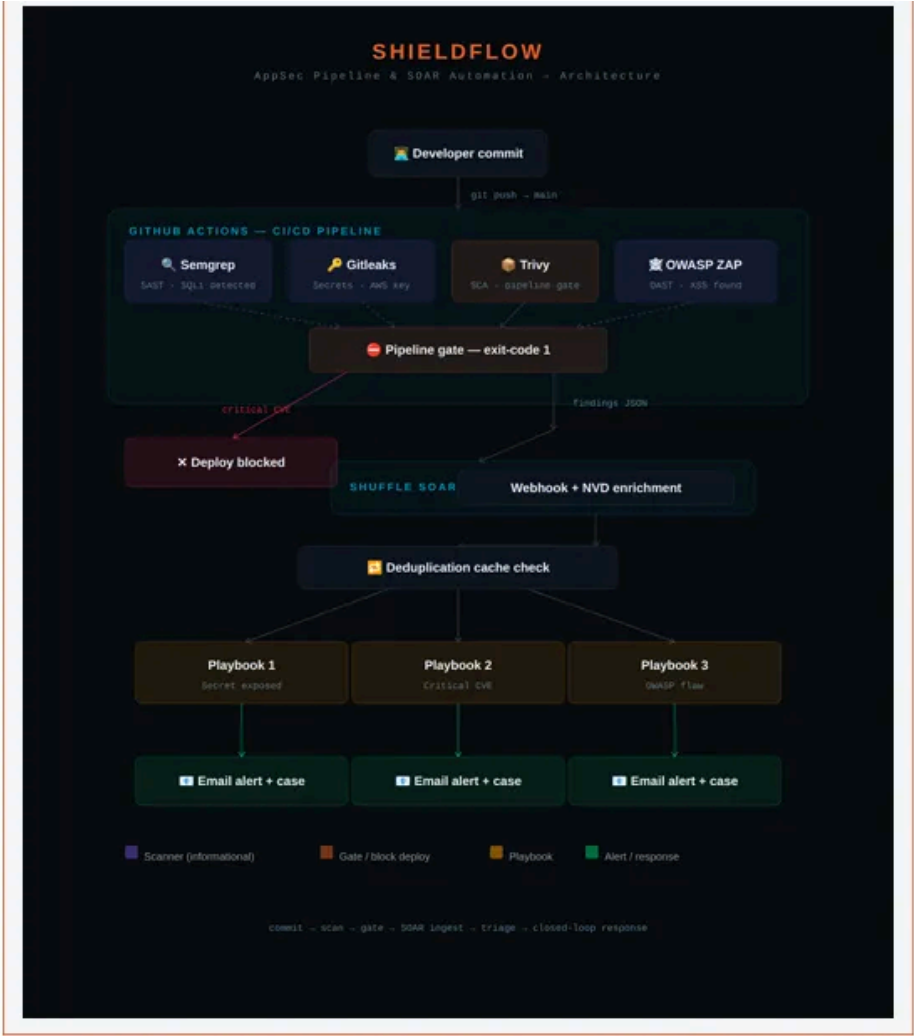
Conclusion

ShieldFlow demonstrates how organizations can integrate security into modern software delivery pipelines using open-source technologies.

By combining SAST, DAST, SCA, secrets detection, CI/CD automation, and SOAR orchestration, we created a platform capable of identifying

vulnerabilities, prioritizing risk, and initiating response actions automatically.

The project highlights the growing importance of DevSecOps practices and shows that effective security automation can be achieved without expensive enterprise tooling.



Evidence: Full pipeline flow — commit → 4 scanners → gate → SOAR ingestion → dedup → 3 playbooks → alerts.

Figure 16: Complete ShieldFlow workflow from code commit to automated security response.



Written by Expadox Lab

17 followers · 1 following

Follow

